

From Chatbots to Agents

Practical workflows for research

OUSSAMA ZEKRI

agent terminal

```
$ latexmk -pdf main.tex
```

fixing undefined reference...

output written to paper.pdf

code

research.pdf

Chatbots vs agents

CHATBOT



ChatGPT



Claude



Gemini

Great at **conversation**:

- explain
- draft
- brainstorm

The user still moves the answer into the work.

AGENT



Cursor



Codex



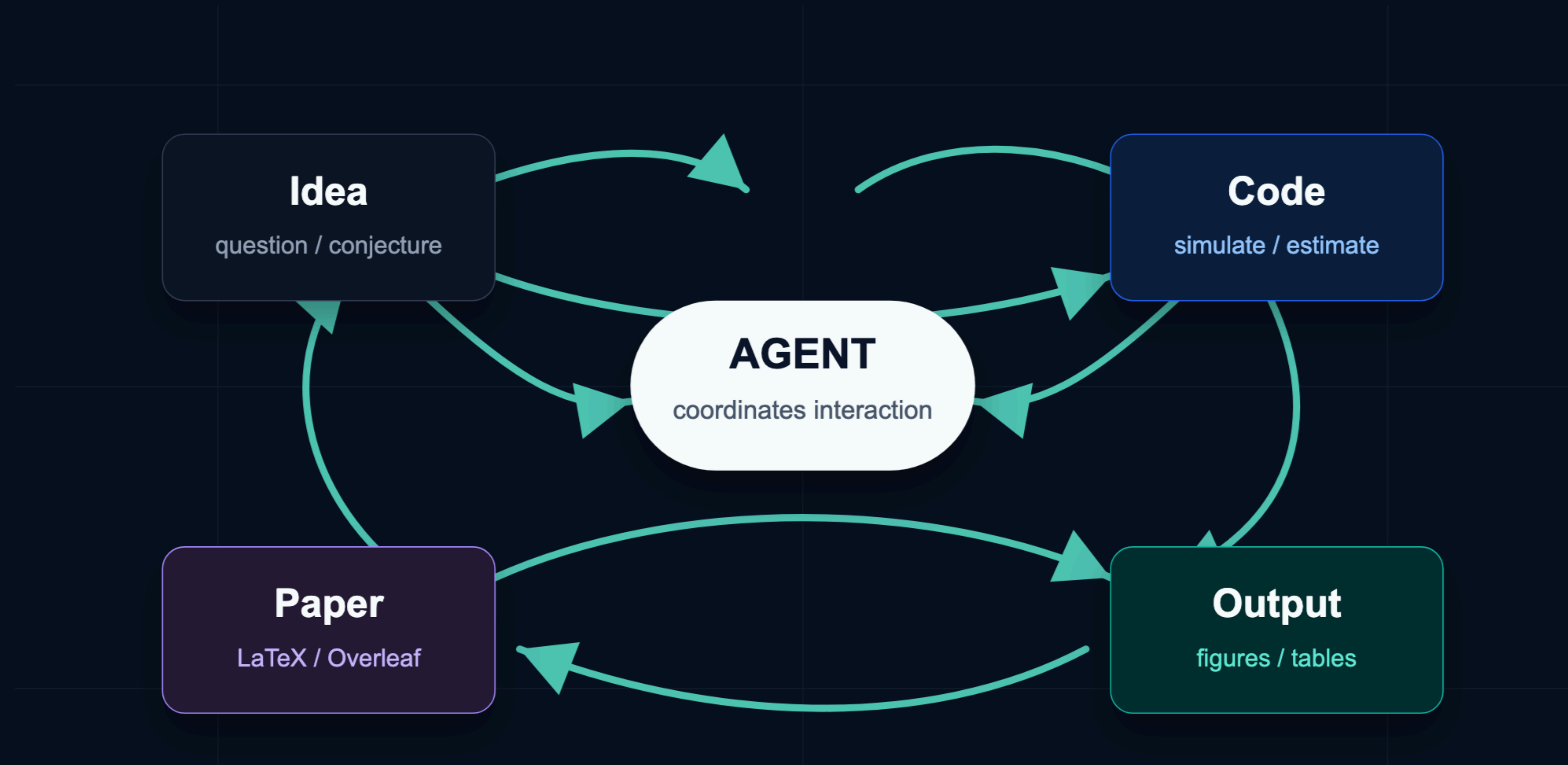
Claude Code

Useful inside **workflows**:

- read files
- edit code
- run commands
- compile, inspect errors, iterate

The difference is not intelligence only. It is **agency inside the workflow**.

Research needs interaction



The point is not only automation: code, outputs, ideas, and paper text need to constantly inform each other.

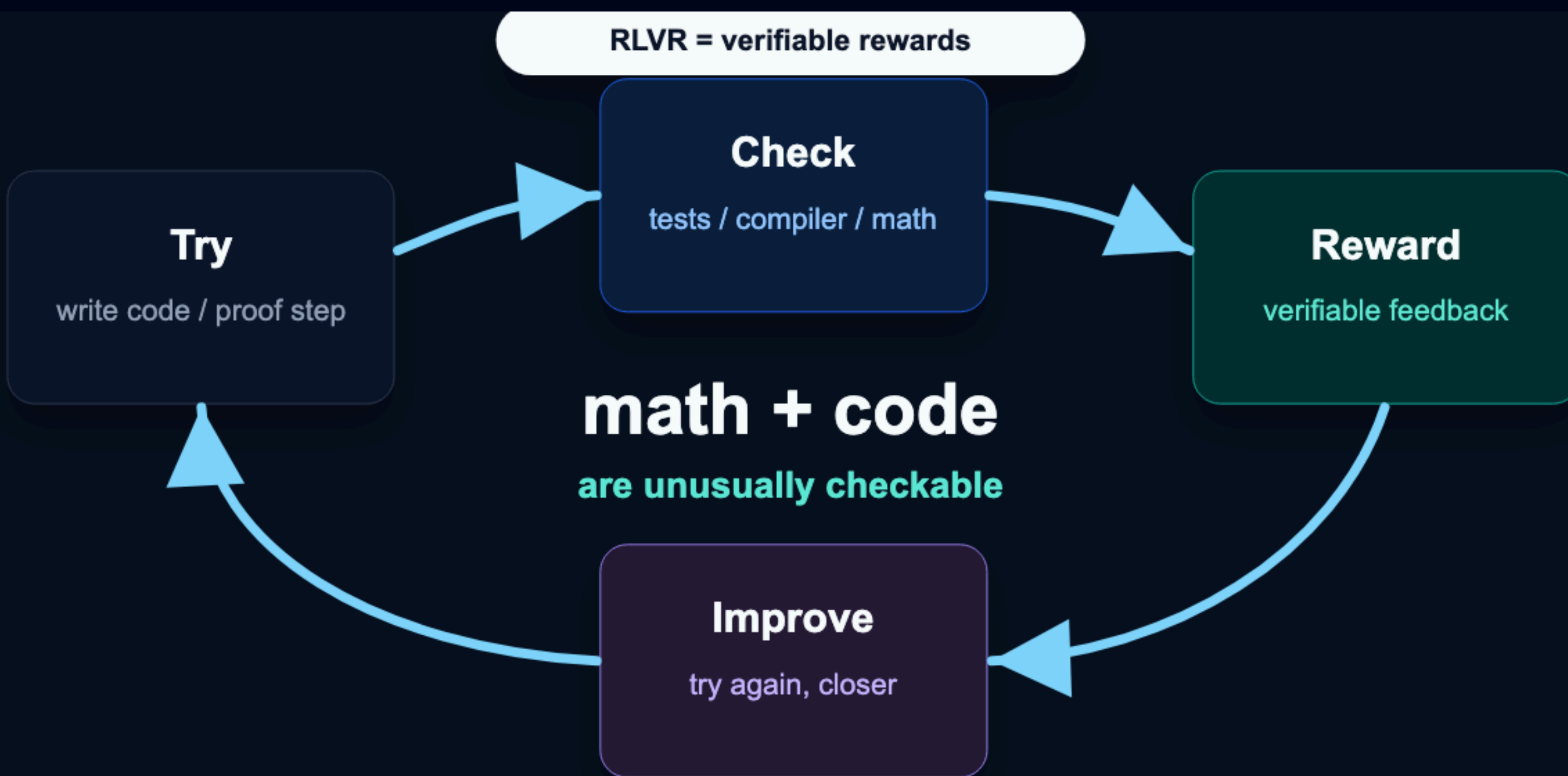
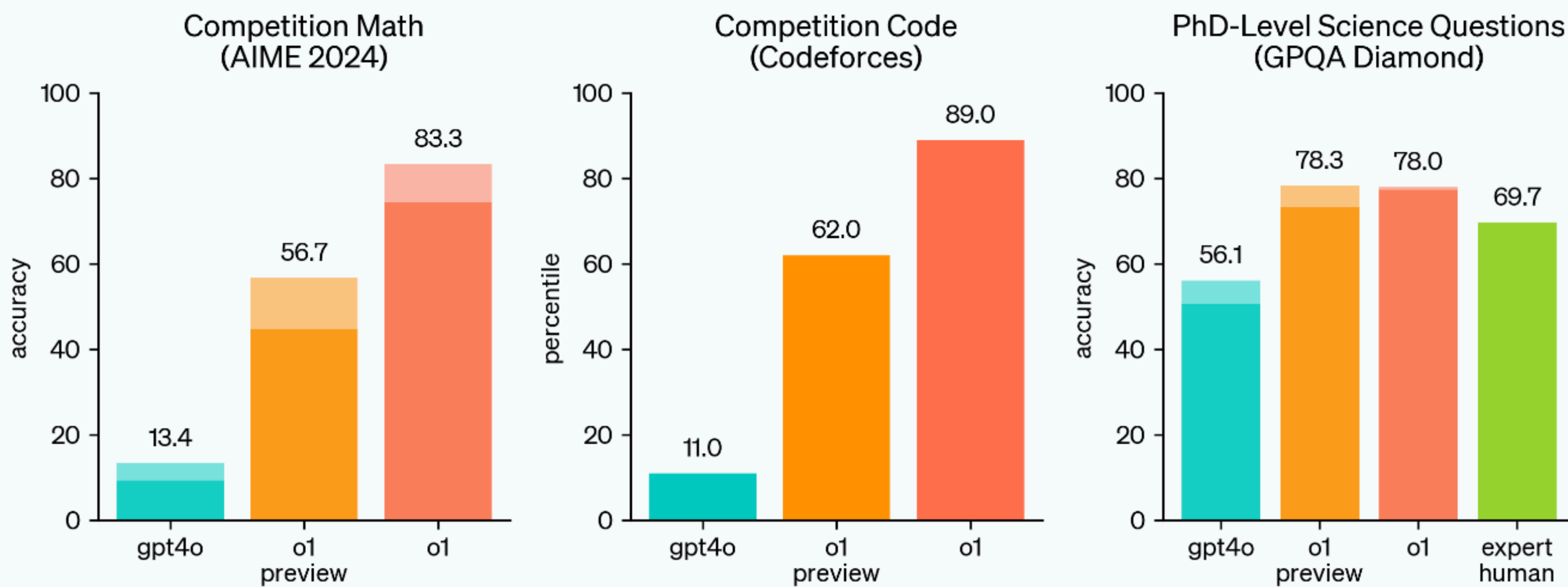
Why models are so strong at math and code

RLVR: REWARD ANSWERS THAT PASS A VERIFIER

$$\max_{\theta} \mathbb{E}_{y \sim \pi_{\theta}(\cdot|x)} [R(x, y)]$$

- x : input problem
- $\pi_{\theta}(y | x)$: model
- y : generated answer
- $R(x, y) \in \{0, 1\}$: automatic verifier

Maximize the probability of answers that pass the verifier.



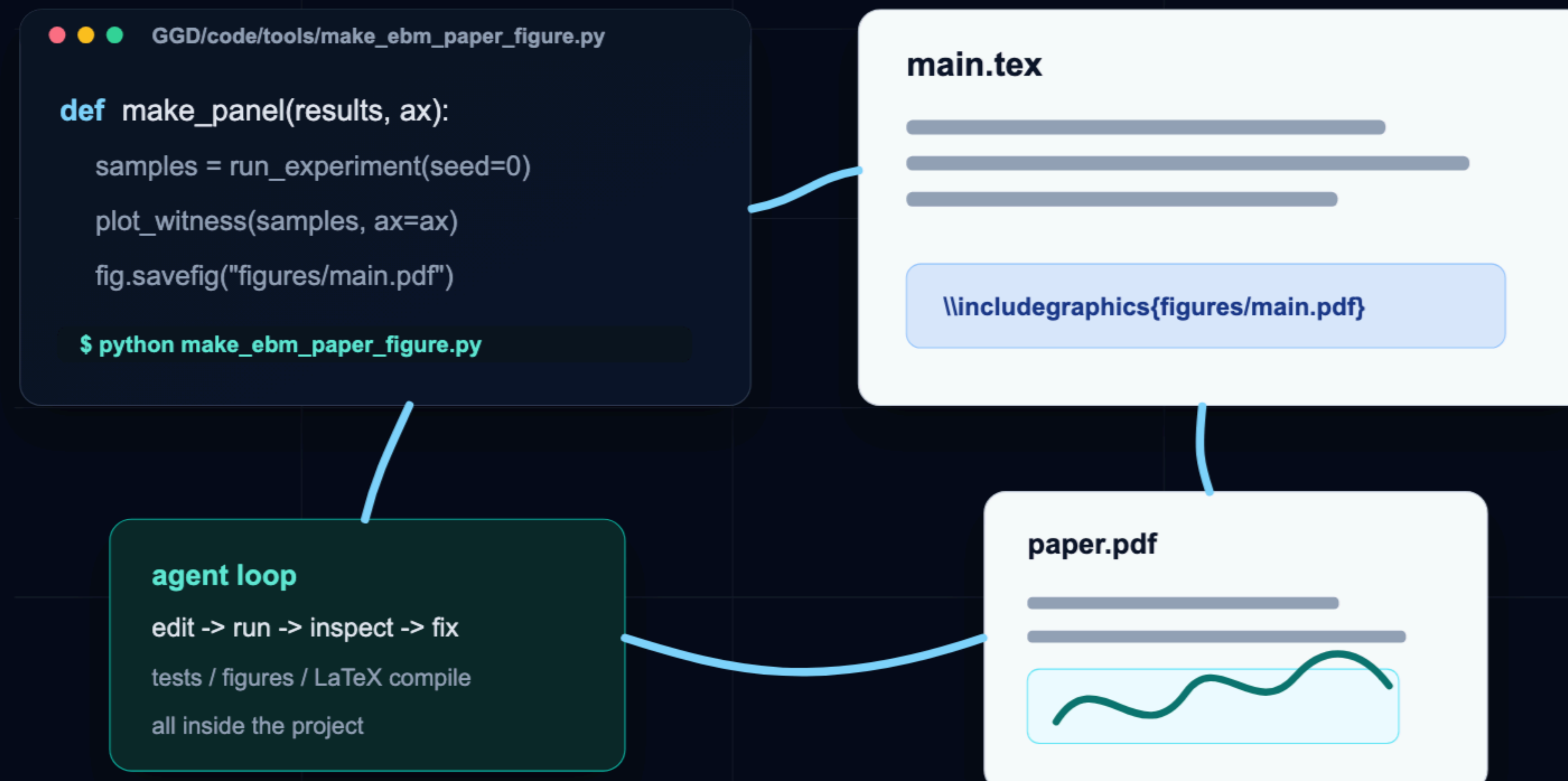
REMARKS ON THE DISPROOF OF THE UNIT DISTANCE CONJECTURE

NOGA ALON, THOMAS F. BLOOM, W. T. GOWERS, DANIEL LITT, WILL SAWIN, ARUL SHANKAR, JACOB TSIMERMAN, VICTOR WANG, AND MELANIE MATCHETT WOOD

ABSTRACT. We present a short, digested, human-verified version of the recent OpenAI-generated counterexample to the Erdős unit distance conjecture, and a sequence of reflections on it. The argument relies crucially on ideas that may, at least in retrospect, be attributed to Ellenberg-Venkatesh, Golod-Shafarevich, and Hajir-Maire-Ramakrishna.

CONTENTS		5. W T Gowers	9	10. Victor Wang	16
1. Introduction	1	6. Daniel Litt	12	11. Melanie Matchett Wood	17
2. Proof and further overview	3	7. Will Sawin	13	References	17
3. Noga Alon	7	8. Arul Shankar	15		
4. Thomas Bloom	7	9. Jacob Tsimerman	16		

Workflow 1: code + paper as one system



WHAT THE AGENT DOES

- reads the repo
- edits code and `.tex`
- runs scripts
- compiles LaTeX
- fixes errors from outputs

Example: from idea to paper output

me: Implement a small simulation for this model.
Generate the figure, insert it in the paper, and compile the PDF.

agent: reading project files...

agent: editing simulation script...

run: figure labels overlap

agent: fixing layout...

latexmk: output written to paper.pdf

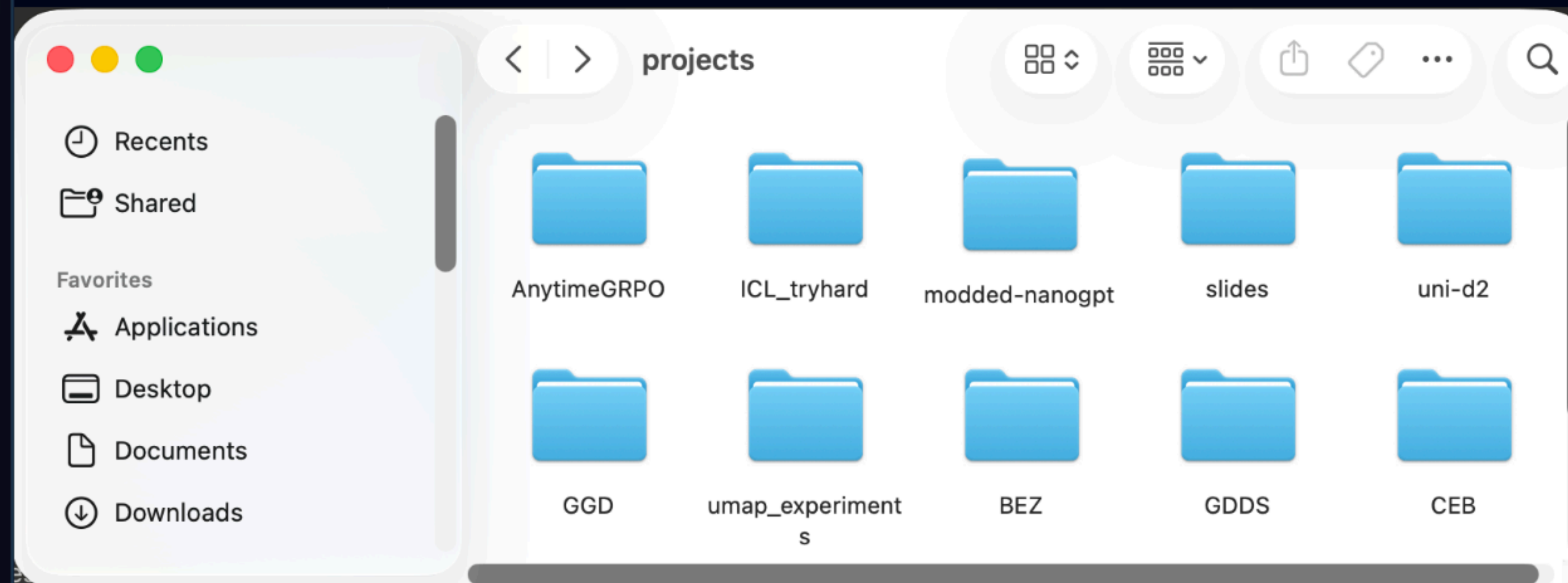
WHAT I SHOW IN PRACTICE

- prompt in the agent
- actual code / LaTeX diff
- generated figure or table
- `latexmk` output
- final PDF excerpt

The value is not a perfect first answer.
The value is closing the loop faster.

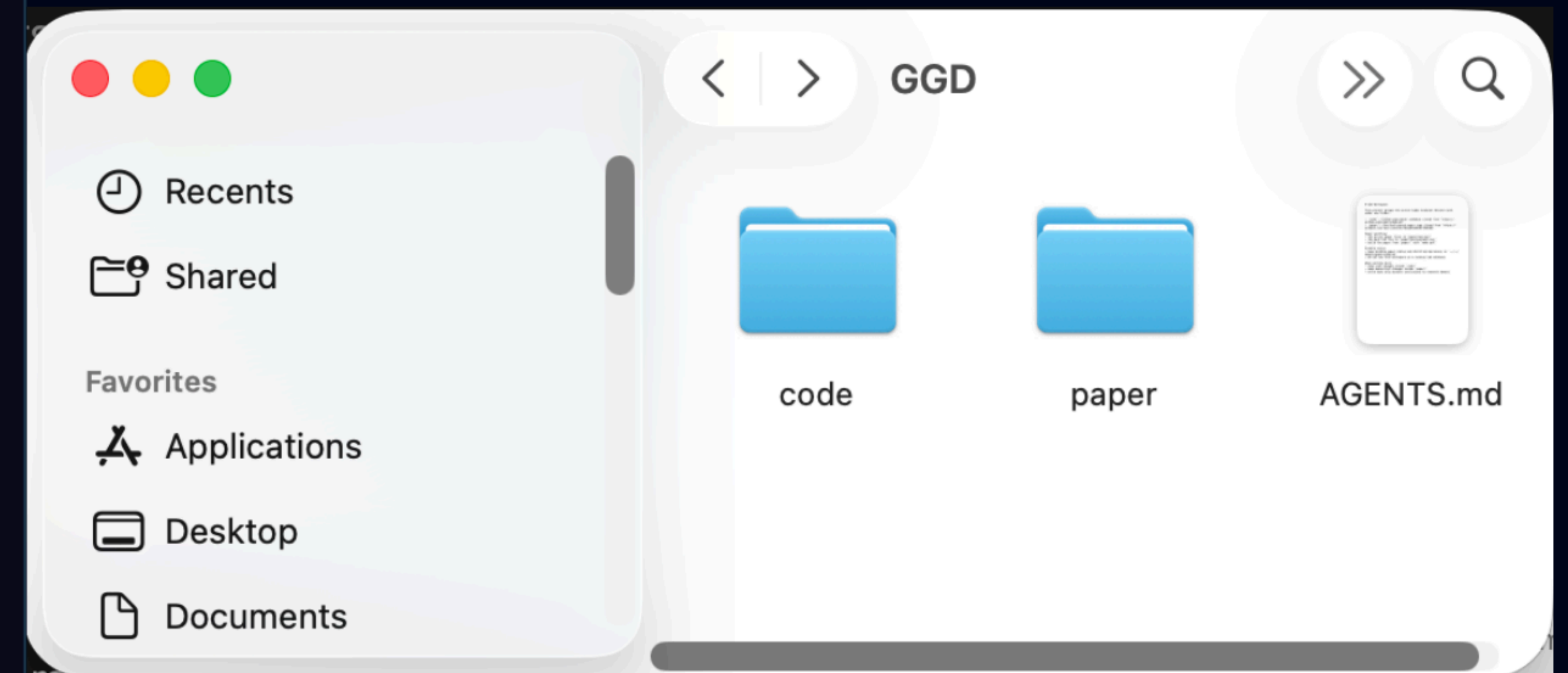
Example: project context for an agent

1. Research projects



Each research project is a folder the agent can inspect.

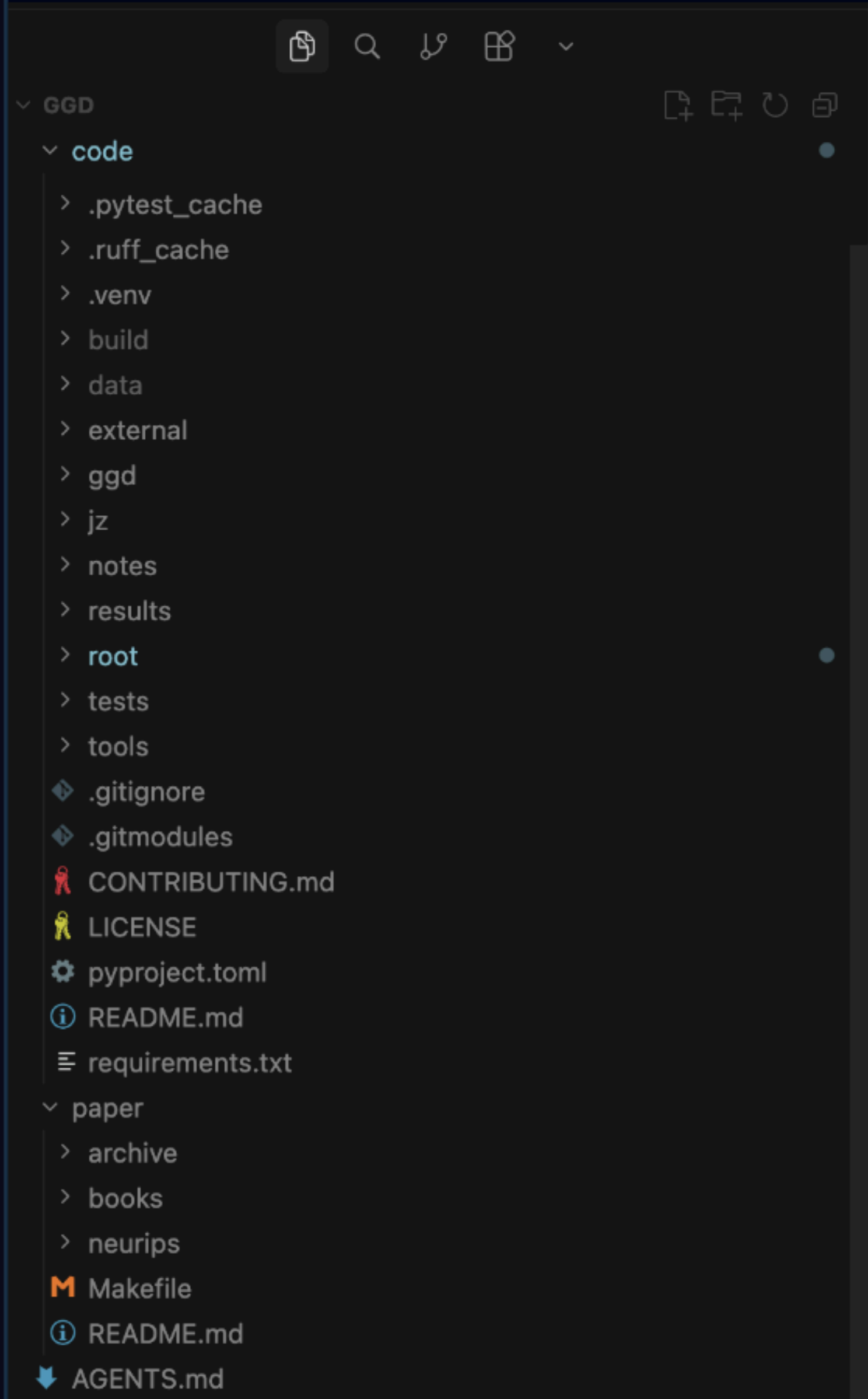
2. One project: GGD



The project exposes its local structure: `code/`, `paper/`, `AGENTS.md`.

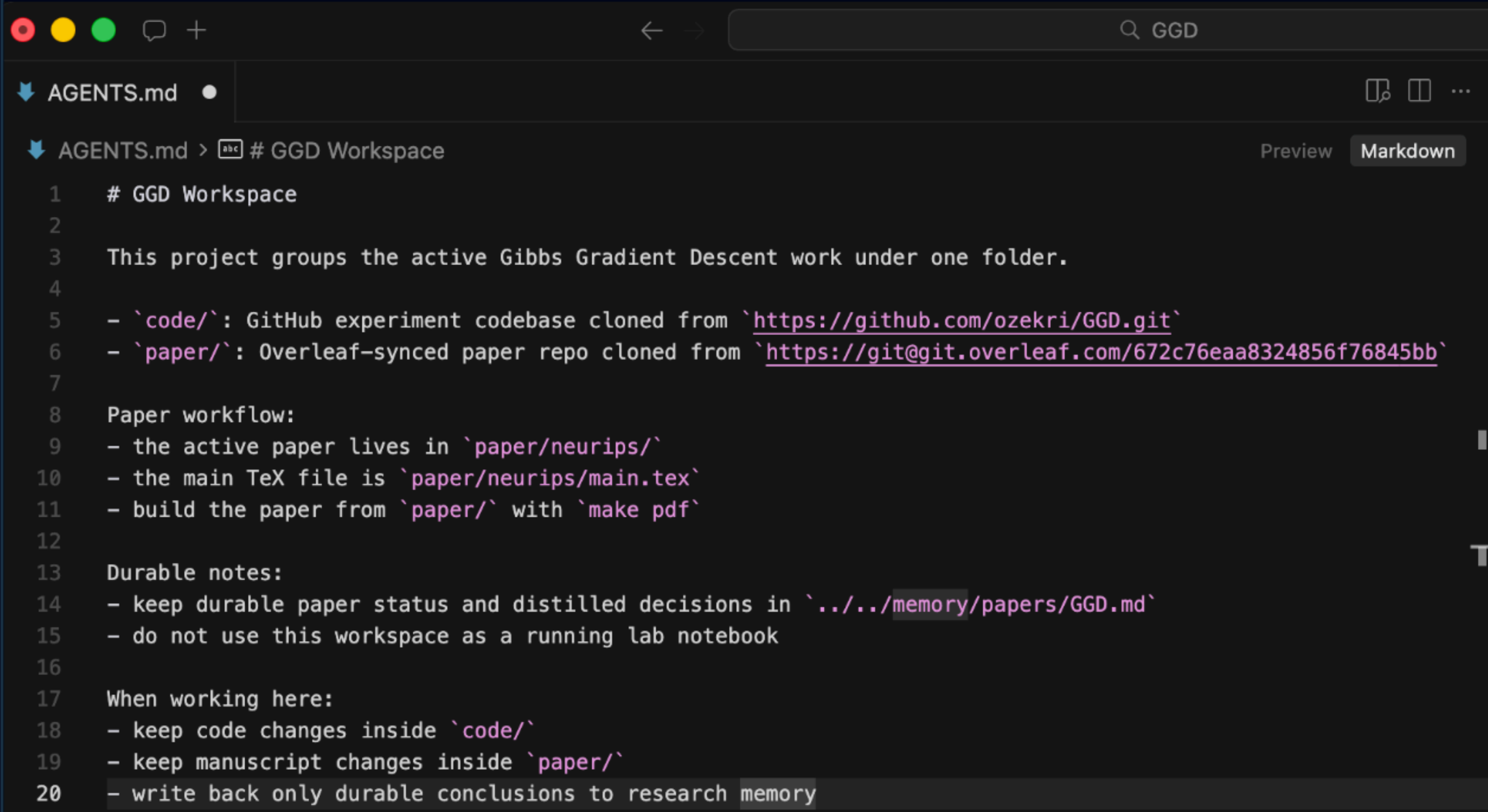
Inside a project: code, paper, instructions

3. Code and paper live together



The agent can move between experiments, figures, tests, and LaTeX.

4. AGENTS.md gives local instructions



Project-specific rules tell the agent how to work safely.

Workflow 2: math concept to video

RESEARCH COMMUNICATION LOOP

- mathematical object
- visual intuition
- storyboard
- Remotion / React code
- render and revise

A creative video task
becomes a code-and-
math task.

Auto-Encoding Variational Bayes

Diederik P. Kingma
Machine Learning Group
Universiteit van Amsterdam
dpkingma@gmail.com

Max Welling
Machine Learning Group
Universiteit van Amsterdam
welling.max@gmail.com

Reparameterization Trick

move randomness outside the network

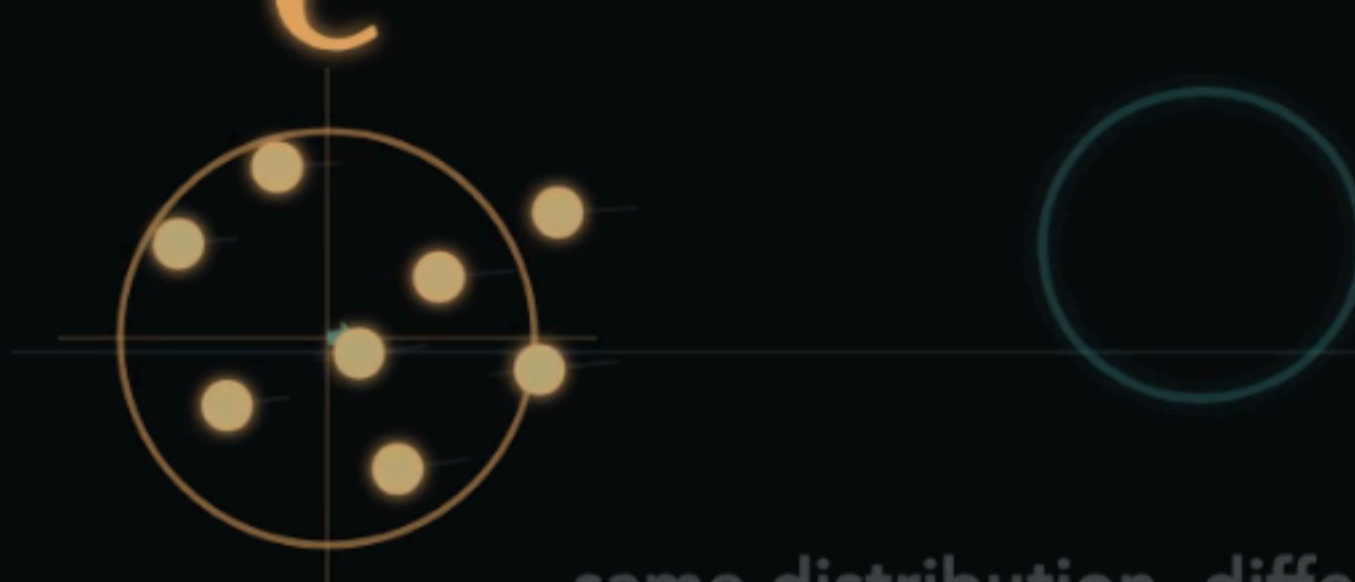
$$z = \mu_{\theta}(x) + \sigma_{\theta}(x) \odot \epsilon, \quad \epsilon \sim \mathcal{N}(0, I)$$

the original problem
sampling blocks gradients

$$z \sim q_{\theta}(z | x) = \mathcal{N}(\mu_{\theta}(x), \text{diag } \sigma_{\theta}^2(x))$$

$$z \stackrel{d}{=} \mu_{\theta}(x) + \sigma_{\theta}(x) \odot \epsilon, \quad \epsilon \sim \mathcal{N}(0, I)$$

fixed noise becomes a trainable sample ∇ through μ, σ



same distribution, differentiable path

trainable path

Example from `tiny-applied-math`: reparameterization trick, generated as an executable animation.

[Open the video](#)

Why Remotion makes this work



MATH FIGURES AND VIDEOS BECOME CODE.

When the medium is React, SVG, CSS, and frames, an agent can write, render, inspect, and revise the explanation.

```
1 const Circle = () => (  
2   <svg viewBox="0 0 200 200">  
3     <circle  
4       cx="100" cy="100" r="70"  
5       fill="#38bdf8"  
6     />  
7   </svg>  
8 );
```

RENDERED FRAME



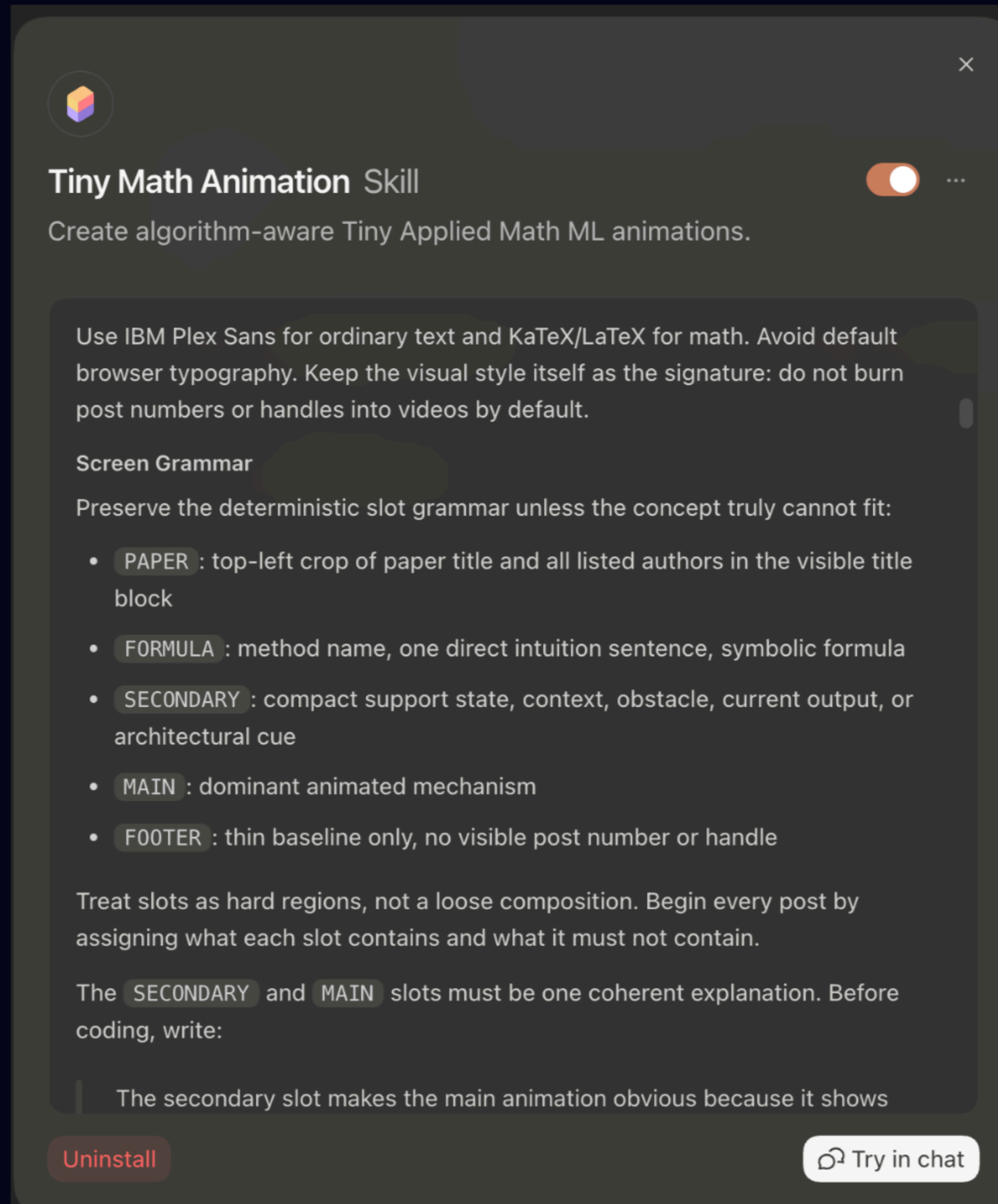
1 Write the figure as code

2 Render frames, figures, or MP4

3 Ask the agent to fix timing, labels, layout

This is powerful because agents are already excellent at code: the video becomes an executable object, not a static design request.

Skills: package precise instructions



A SKILL IS A REUSABLE INSTRUCTION SET FOR THE AGENT.

- write the visual grammar once
- specify rendering and checking steps
- encode what “good” output means
- reuse the same workflow across concepts

Instead of prompting from scratch, I give the agent detailed instructions and let it execute the workflow.



Skill Creator

Create or update a skill



The agent can write the skill too.

Delegate the loop, own the claims

✓ ship with the agent

INSPECTABLE WORK PRODUCTS

- ✓ boilerplate code and refactors
- ✓ debugging traces and logs
- ✓ LaTeX cleanup and paper plumbing
- ✓ figure iterations
- ✓ end-to-end animation and slides

✗ own myself

SCIENTIFIC JUDGMENT

- ! theorem statements
- ! interpretation of evidence
- ! identification and causality
- ! citations and attributions
- ! final claims

The agent makes iteration cheap; I still decide what is true.

The useful split is not “trust or distrust”; it is deciding which work products can be inspected and which claims must be owned.

Takeaways

1

Agents act on files, code, tools, and outputs.

2

Research production is a loop: **code** → **result** → **paper** → **revise**.

3

You can personalize this to **your own workflow** through skills and clever orchestration.

Agents do not replace research judgment (yet!). They change the interface through which we can do research work.